

Marker guide

Aligned with `data-analyst-interviewee.md`. Keep that file open for full stems and sample tables.

Format: Questions are sent ahead. In the interview the candidate presents and talks through each answer.

Candidate brief (same as their title page): Prepare ahead; present and talk through each answer; SQL where asked; short explanations where asked; order as given. If they used external resources (search, docs, AI, etc.) while preparing, they should say what they did not know and how they found it.

Exact wording is not required. Judge the substance of what they say (and any notes they share).

Total /35.

Marks scale (every question)

Score	Meaning
0	No relevant points
1	Some relevant points, but core missing or wrong
3	Core correct; secondary detail missing or wrong
5	Core and secondary detail covered

Do not use 2 or 4. Only 0, 1, 3, or 5.

Marks overview

Q	Topic	Marks
1	Two left joins	5
2a	Most recent order	5
2b	Second most recent order	5
3a	Accounts per tier per day	5
3b	Silver to gold (past 30 days)	5
4	Git	5
5	Dashboard request	5
	Total	35

1. Two left joins

As presented: compare A (`WHERE` on `order_date`) vs B (date in `ON`) on `customers` and `orders` . Full SQL on the interviewee slide.

Core

- A and B can differ.
- A drops customers with no matching 2024+ order.
- B keeps those customers.

Secondary

- On B, those customers have null order columns.
- Join / `ON` before `WHERE` , or `WHERE` on order fields undoes the left join.
- Several matching orders → several rows (optional strength).

Score	When
0	No useful distinction
1	Mentions joins/filters but wrong or incomplete on who is kept/dropped
3	Core (A drops / B keeps) correct; thin on why or nulls
5	Core plus mechanism (and ideally nulls / 1:many)

1. Model explanation

```
-- A: LEFT JOIN then WHERE o.order_date >= '2024-01-01'  
-- B: LEFT JOIN ... ON key AND o.order_date >= '2024-01-01'
```

1. Left-join `customers` to `orders` .
2. A: date filter in `WHERE` after the join.
3. B: date filter in `ON` .

No orders: left join keeps the customer with null `order_date` .

- A: null fails `WHERE` → removed.
- B: kept, null `order_id` .

Only 2023 orders: same as no orders for this filter.

Several 2024 orders: both return several rows.

2a. Most recent order

As presented: `orders` ; identity `order_id` ; all columns not nullable; return four columns; sample rows on interviewee slide.

Core

- One row per customer from `orders` .
- Most recent = highest `order_id` .
- Returns the full order row (`customer_id` , `order_id` , `order_date` , `amount`).
- Columns are not nullable (no null-handling required).

Secondary

- Clear complete SQL.
- Matches example: customer 1 → 13, customer 2 → 14.

Allowed for 5: `ROW_NUMBER` or `MAX(order_id)` + join back.

`MAX(order_id)` alone (ids only) → at most **1** (relevant but core incomplete).

Score	When
0	Irrelevant
1	Some SQL toward latest, but wrong grain/columns/oldest
3	Core correct; messy SQL or fails example check
5	Core + working query (partition preferred; max+join OK)

2a. Model — preferred

```
;WITH ranked AS (  
    SELECT customer_id, order_id, order_date, amount,  
           ROW_NUMBER() OVER (  
               PARTITION BY customer_id  
               ORDER BY order_id DESC  
           ) AS rn  
    FROM orders  
)  
SELECT customer_id, order_id, order_date, amount  
FROM ranked  
WHERE rn = 1;
```

2a. Model — also valid for 2a only

```
;WITH latest AS (  
  SELECT customer_id, MAX(order_id) AS order_id  
  FROM orders  
  GROUP BY customer_id  
)  
SELECT o.customer_id, o.order_id, o.order_date, o.amount  
FROM orders AS o  
INNER JOIN latest AS l  
  ON o.customer_id = l.customer_id  
  AND o.order_id = l.order_id;
```

Does **not** unlock 2b without a new approach.

2b. Second most recent order

As presented: same `orders` table; second most recent; exclude customers with fewer than two orders.

Core

- Second-highest `order_id` per customer (e.g. `rn = 2`).
- Same result columns as 2a.
- Customers with one order excluded.

Secondary

- Same ranking pattern as 2a, only the filter changes.
- Matches example: customer 1 → 11, customer 2 → 12.

`MAX` / `GROUP BY` **alone** → 0 (does not solve second most recent).

Score	When
0	Missing, or only repeats 2a / max-only
1	Attempts "second" but logic wrong
3	Core correct; weak SQL or example off
5	Core + clean adaptation (typically <code>rn = 2</code>)

2b. Model answer

```
;WITH ranked AS (  
    SELECT customer_id, order_id, order_date, amount,  
           ROW_NUMBER() OVER (  
               PARTITION BY customer_id  
               ORDER BY order_id DESC  
           ) AS rn  
    FROM orders  
)  
SELECT customer_id, order_id, order_date, amount  
FROM ranked  
WHERE rn = 2;
```

3a. Accounts per tier per day

As presented: `account_tier_scd` + `dim_date`; counts per tier per day in 2024; include days with a count of zero. Sample on interviewee slide.

Core

- For each date in 2024, count accounts on each tier that day.
- Join `dim_date` to `account_tier_scd` on validity range.
- Handle null `valid_to` as current.

Secondary

- **LEFT JOIN** from `dim_date` so dates with no matches still appear (`account_cnt` = 0; `tier` may be null). Full date × all-tiers densify is not required unless they choose it.
- Half-open bounds (`D >= valid_from` and `D < valid_to` or null end).
- `GROUP BY` date, tier; filter year 2024 explicitly.
- Sample check: 2024-04-01 → silver 2, gold 1.

Score	When
0	Irrelevant or wrong problem
1	Mentions dates/tiers but no working range join / current handling
3	Core range join + counts; missing left join / zeros / fine bounds
5	Core + left join from date, null current, 2024, correct grain

3a. Model answer

```
SELECT
    d.[date],
    a.tier,
    COUNT(a.account_id) AS account_cnt
FROM dim_date AS d
LEFT JOIN account_tier_scd AS a
    ON d.[date] >= a.valid_from
    AND (a.valid_to IS NULL OR d.[date] < a.valid_to)
WHERE d.[date] >= '2024-01-01'
    AND d.[date] < '2025-01-01'
GROUP BY d.[date], a.tier;
```

3b. Silver to gold

As presented: `account_tier_scd`; silver → gold in the past 30 days; return `account_id`, `changed_at` (gold `valid_from`). Sample rows are structural only — they need not fall in the 30-day window.

Core

- Detect a move from silver to gold on consecutive versions for an account.
- Return `account_id` and `changed_at` (gold `valid_from`).
- Limit to the past 30 days.

Secondary

- `LAG`, `LEAD`, or self-join on consecutive versions.
- Clear filter on both from-tier and to-tier.
- Careful day-over-day spine is allowed (see note); “ever silver then gold” is not.

Score	When
0	Missing, or illegal spine (“ever silver and gold”)
1	Some upgrade idea; not consecutive silver→gold
3	Core consecutive silver→gold + 30 days; rough SQL
5	Core + clean LAG, LEAD, self-join, or valid day-over-day spine

3b. Model — LAG or LEAD

LAG (on the gold row): previous tier + current `valid_from` as `changed_at`.

```
;WITH ordered AS (  
    SELECT account_id, tier, valid_from,  
           LAG(tier) OVER (  
               PARTITION BY account_id ORDER BY valid_from  
           ) AS prev_tier  
    FROM account_tier_scd  
)  
SELECT account_id, valid_from AS changed_at  
FROM ordered  
WHERE prev_tier = N'silver' AND tier = N'gold'  
      AND valid_from >= DATEADD(day, -30, CAST(GETDATE() AS date));
```

LEAD needs two windows: you sit on silver, so you need `LEAD(tier)` and `LEAD(valid_from)` for gold's start date. LAG only needs one because `changed_at` is already on the gold row.

```
;WITH ordered AS (  
    SELECT account_id, tier, valid_from,  
           LEAD(tier) OVER (PARTITION BY account_id ORDER BY valid_from) AS next_tier,  
           LEAD(valid_from) OVER (PARTITION BY account_id ORDER BY valid_from) AS next_from  
    FROM account_tier_scd  
)  
SELECT account_id, next_from AS changed_at  
FROM ordered  
WHERE tier = N'silver' AND next_tier = N'gold'  
      AND next_from >= DATEADD(day, -30, CAST(GETDATE() AS date));
```

3b. Model — self-join

Consecutive versions: gold `valid_from` equals silver `valid_to` (half-open).

```
SELECT
    s.account_id,
    g.valid_from AS changed_at
FROM account_tier_scd AS s
INNER JOIN account_tier_scd AS g
    ON g.account_id = s.account_id
    AND s.tier = N'silver'
    AND g.tier = N'gold'
    AND g.valid_from = s.valid_to
WHERE g.valid_from >= DATEADD(day, -30, CAST(GETDATE() AS date));
```

3b. Date spine — legal vs not

Legal: day-over-day move — tier on day `D` is gold, tier on `D - 1` is silver, `D` in the past 30 days; `changed_at = D` (matches gold `valid_from` for half-open SCD).

Not legal: ever silver and ever gold in the window; currently gold and silver sometime in history; any move into gold; 3a-style counts without a consecutive transition.

Prefer LAG / LEAD / self-join for clarity; accept a careful day-over-day spine if the SQL clearly does that.

4. Git

Core: workflow `pull` / `checkout` / `add` / `commit` / `push` ; conflict = overlapping edits; resolve = choose or edit to a safe final state, then complete the merge.

Secondary: `merge` vs `revert` clear.

Score	When
0	No useful Git knowledge
1	A few commands roughly right; conflict/resolve missing or wrong
3	Core workflow + conflict/resolve basically right; thin on merge/revert
5	Core + merge/revert clear + safe resolve

4. Git — command answers

Command	Enough (for marks)	Stronger
<code>pull</code>	Get remote changes into this branch	Fetch from remote + integrate (merge/rebase) into current branch
<code>checkout</code>	Switch branch / restore files	Moves <code>HEAD</code> to another branch (or commit); updates working tree
<code>add</code>	Stage for next commit	Copies changes into the index; branch pointer unchanged
<code>commit</code>	Save local snapshot	New commit object; advances current branch tip; <code>HEAD</code> follows
<code>push</code>	Send commits to remote	Updates remote branch ref (e.g. <code>origin/main</code>) to match local
<code>merge</code>	Combine another branch	Joins histories; fast-forward moves branch tip, or creates merge commit
<code>revert</code>	Undo with a new commit	New commit reverses a prior diff; history kept (unlike reset)

`origin` = usual remote name. Remote-tracking branches (e.g. `origin/main`) record where the remote tip was last seen.

4. Git — conflict model

Cause

Branches A and B both have commits that change the **same lines** of the same file (or one deletes what the other edits). When you merge (or pull that merges), Git cannot choose which change to keep, so it stops and marks a **conflict**.

Resolve

1. Open the conflicted file and find the conflict markers.
2. Choose one side, the other, or edit to a combined final text that is correct and safe.
3. Complete the merge with a commit.

5. Dashboard request

As presented: A request comes in for a new dashboard. What questions do you ask, and what information do you need, before you start working?

Score 0 / 1 / 3 / 5 from how many **areas** are covered (breadth of topics; depth per topic not required).

Areas (1+ point of substance each counts as covering that area)

Area	Examples that count
Existing estate	Other reports/dashboards; docs; policy; process; standards; "what we already have or do"
Requirements precision	Exact metrics; definitions/terminology; calculation rules; grain; filters; acceptance / "done"
Data sources	Where data lives; systems/tables; access; freshness/latency; quality or known gaps
Purpose & use	Why; who / audience; use case; decisions or actions taken from it
Delivery & tech	Tool/technology; timing; refresh; access; deployment / how it is delivered

Score	When
0	Would just build / nothing useful
1	Only one area, or vague "ask for more detail"
3	Two areas with real substance
5	Three or more areas covered

Prefer breadth of topics (including **precise requirements** and **data sources**) over long depth on one theme or tool choice alone.

Score sheet

Q	Max	Awarded (0/1/3/5)	Notes
1	5		
2a	5		
2b	5		
3a	5		
3b	5		
4	5		
5	5		
Total	35		